Java OOP Interview Guide

Concepts, code examples, real-world case studies
and the top 50 interview questions

REAL-WORLD EXAMPLES: UBER | E-COMMERCE | NETFLIX

154 core concepts explained with code

3 full real-world case studies

50 top interview questions + output-prediction puzzles

Quick-revision cheat sheets

Contents

Part 1: OOP Fundamentals . . . 3
Part 2: Classes, Objects and Constructors . . . 8
Part 3: Access Modifiers and Keywords . . . 10
Part 4: Abstract Classes and Interfaces . . . 12
Part 5: Polymorphism, Binding and Object Behavior . . . 15
Part 6: Object Class Methods . . . 17
Part 7: Exception Handling (OOP Context) . . . 19
Part 8: Java-Specific OOP Concepts . . . 21
Part 9: Generics and Collections . . . 25
Part 10: SOLID Principles and Design Patterns . . . 27
Part 11: Memory, Garbage Collection and OOP Internals . . . 32
Part 12: Miscellaneous and Tricky OOP Questions . . . 33
Part 13: Modern Java Features (Java 8 to 17) . . . 37
Part 14: Quick Revision Cheat Sheets . . . 38
Part 15: Real-World Case Studies . . . 39
Part 16: Top 50 Java OOP Interview Questions . . . 43

How to use this guide

Each concept is numbered (Q1 to Q154) in the same order as your notes, followed by a code example and, for the major ideas, a green real-world box (Uber, E-commerce, Netflix). Part 14 has revision cheat sheets, Part 15 builds three complete mini-systems, and Part 16 closes with the top 50 interview questions and output-prediction puzzles. Code uses Java 8+ syntax; a few lines use Java 16/17 features (records, pattern matching) and are marked.

Part 1: OOP Fundamentals

Q1. What is Object-Oriented Programming (OOP)?

OOP is a programming paradigm based on the concept of "objects," which bundle data (fields) and behavior (methods) together. It helps organize code into reusable, modular units that model real-world entities.

> **Real-world: Uber**
> In Uber, everything is an object: a `Rider`, a `Driver`, a `Ride`, a `Vehicle`, a `Payment`. Each one has data (name, rating, location) and behavior (requestRide, acceptRide, endRide).

```
class Ride {
    String pickup, drop;
    double fare;

    void start() { System.out.println("Ride started: " + pickup + " -> " + drop); }
    void end()   { System.out.println("Ride ended. Fare = " + fare); }
}
```

Q2. What are the four main pillars of OOP?

Encapsulation, Inheritance, Polymorphism, and Abstraction.

Pillar	One-line meaning	Uber example
Encapsulation	Hide data, expose controlled access	Wallet balance is private; changed only via addMoney()/pay()
Inheritance	Child reuses parent's code	Car, Auto, Bike all extend Vehicle
Polymorphism	Same call, different behavior	calculateFare() differs for Auto vs Premium Car
Abstraction	Show what, hide how	Rider taps "Book"; driver-matching algorithm stays hidden

Q3. What is a Class in Java?

A class is a blueprint or template that defines the properties (fields) and behaviors (methods) of the objects that it creates.

Q4. What is an Object in Java?

An object is an instance of a class. It has its own state (values of fields) and can perform actions using the methods defined in its class.

```
public class Main {
    public static void main(String[] args) {
        Ride r1 = new Ride();           // object 1
        r1.pickup = "Gandhipuram"; r1.drop = "Airport"; r1.fare = 450;

        Ride r2 = new Ride();           // object 2 - independent state
        r2.pickup = "RS Puram";   r2.drop = "PSG iTech"; r2.fare = 120;

        r1.start();                     // Ride started: Gandhipuram -> Airport
        r2.start();                     // Ride started: RS Puram -> PSG iTech
    }
}
```

Q5. What is the difference between a Class and an Object?

A class is a definition or template; it does not occupy memory on its own. An object is a real instance created from that class, which occupies memory and holds actual data.

> Analogy: `Ride` (class) is the "ride booking form". Each filled-in form (r1, r2) is an object.

Q6. What is Encapsulation?

Encapsulation is the practice of bundling data (fields) and methods into a single unit (class), while restricting direct access to some of the object's components using access modifiers like `private`.

Q7. Why is Encapsulation important?

It protects data from unauthorized access and modification, improves maintainability, and allows internal implementation details to change without affecting external code.

Q8. How is Encapsulation achieved in Java?

By declaring fields as private and providing public getter and setter methods to access and modify them.

> **Real-world: Uber Wallet**
> Nobody should be able to write `wallet.balance = 1000000`. The balance is private and can only change through validated methods.

```
class Wallet {
    private double balance;                        // hidden data

    public double getBalance() { return balance; }

    public void addMoney(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public void pay(double amount) {
        if (amount > balance) throw new IllegalStateException("Insufficient balance");
        balance -= amount;
    }
}
// wallet.balance = 1_000_000;   // COMPILE ERROR - private access
```

Q9. What is Abstraction?

Abstraction is the concept of hiding complex implementation details and showing only the essential features of an object to the user.

Q10. How is Abstraction different from Encapsulation?

Abstraction focuses on hiding complexity by showing only relevant details (what an object does), while encapsulation focuses on hiding data by restricting direct access (how the data is protected).

Aspect	Abstraction	Encapsulation
Focus	What an object does	How data is protected
Level	Design level	Implementation level
Achieved by	Abstract classes, interfaces	private fields + getters/setters
Example	PaymentMethod.pay()	Wallet.balance is private

Q11. How is Abstraction achieved in Java?

Using abstract classes and interfaces.

> **Real-world: E-commerce checkout**
> The checkout code only knows `PaymentMethod.pay()`. Whether it is UPI, a card or Cash on Delivery is hidden behind the abstraction.

```
interface PaymentMethod { boolean pay(double amount); }

class UpiPayment implements PaymentMethod {
    public boolean pay(double amount) { System.out.println("Paid " + amount + " via UPI"); return true; }
```

```
}
class CardPayment implements PaymentMethod {
    public boolean pay(double amount) { System.out.println("Paid " + amount + " via Card"); return true; }
}

class Checkout {
    void placeOrder(PaymentMethod method, double amount) {   // depends on abstraction
        method.pay(amount);
    }
}
```

Q12. What is Inheritance?

Inheritance is a mechanism where one class (subclass/child) acquires the properties and behaviors of another class (superclass/parent), promoting code reuse.

```
class Vehicle {
    String plateNumber;
    void start() { System.out.println(plateNumber + " engine started"); }
}
class Car  extends Vehicle { int seats = 4; }     // reuses plateNumber + start()
class Bike extends Vehicle { boolean helmetProvided = true; }

Car c = new Car();
c.plateNumber = "TN-37-AB-1234";
c.start();                                        // inherited from Vehicle
```

Q13. What are the types of Inheritance supported in Java?

Single, Multilevel, and Hierarchical inheritance are supported directly. Multiple inheritance (of classes) is not supported directly but can be achieved through interfaces.

```
class A {}
class B extends A {}            // Single:       A -> B
class C extends B {}            // Multilevel:   A -> B -> C
class D extends A {}            // Hierarchical: A -> B and A -> D

interface Flyable {}  interface Swimmable {}
class Duck implements Flyable, Swimmable {}   // "multiple inheritance" via interfaces
```

Q14. Why doesn't Java support multiple inheritance with classes?

To avoid the "Diamond Problem," where ambiguity arises if two parent classes have methods with the same signature and the compiler cannot determine which one the subclass should inherit.

```
class Camera { void start() { System.out.println("Camera on"); } }
class Phone  { void start() { System.out.println("Phone on");  } }

// class SmartPhone extends Camera, Phone {}   // COMPILE ERROR
// new SmartPhone().start();  -> which start()? Camera's or Phone's? Ambiguous!
```

Q15. What is Polymorphism?

Polymorphism means "many forms." It allows an object to take on different behaviors depending on the context, typically through method overloading and method overriding.

Q16. What are the types of Polymorphism in Java?

- **Compile-time (Static) Polymorphism** - achieved through method overloading.
- **Runtime (Dynamic) Polymorphism** - achieved through method overriding.

Q17. What is Method Overloading?

Method overloading occurs when multiple methods in the same class have the same name but different parameter lists (different number, type, or order of parameters).

> **Real-world: E-commerce search**
> One `search` name, different inputs: keyword only, keyword + category, keyword + price range.

```
class ProductSearch {
    String search(String keyword) {
        return "Results for " + keyword;
```

```
    }
    String search(String keyword, String category) {
        return "Results for " + keyword + " in " + category;
    }
    String search(String keyword, double minPrice, double maxPrice) {
        return "Results for " + keyword + " between " + minPrice + " and " + maxPrice;
    }
}
```

Q18. What is Method Overriding?

Method overriding occurs when a subclass provides its own implementation of a method that is already defined in its superclass, with the same name, return type, and parameters.

> **Real-world: Uber fare**
> Every ride type calculates fare differently, but the app calls the same `calculateFare()` on a `Ride` reference. The actual object decides which version runs.

```
class Ride {
    double calculateFare(double km) { return km * 10; }
}
class AutoRide extends Ride {
    @Override double calculateFare(double km) { return 30 + km * 12; }
}
class PremiumRide extends Ride {
    @Override double calculateFare(double km) { return 100 + km * 25; }
}

Ride ride = new PremiumRide();                // reference: Ride, object: PremiumRide
System.out.println(ride.calculateFare(10));   // 350.0  -> PremiumRide's version (runtime)
```

Q19. What is the difference between Method Overloading and Method Overriding?

Overloading happens within the same class and is resolved at compile time. Overriding happens between a superclass and subclass and is resolved at runtime.

Aspect	Overloading	Overriding
Where	Same class (or subclass)	Parent and child class
Parameters	Must differ	Must be the same
Return type	Can differ	Same or covariant
Resolved	Compile time (static binding)	Runtime (dynamic binding)
Access modifier	Any	Cannot be more restrictive
static/private/final	Allowed	Not allowed to override

Q20. Can we overload the main() method in Java?

Yes, a class can have multiple overloaded main() methods, but the JVM will only call the standard `public static void main(String[] args)` as the entry point.

```
public class Demo {
    public static void main(String[] args) {      // JVM calls only this one
        System.out.println("Standard main");
        main(5);                                  // we can call the others manually
    }
    public static void main(int x) { System.out.println("Overloaded main: " + x); }
}
```

Q21. Can we override a static method in Java?

No, static methods belong to the class, not to an instance, so they cannot be overridden. If a subclass defines a static method with the same signature, it is called method hiding, not overriding.

Q22. What is Method Hiding?

When a subclass defines a static method with the same signature as a static method in its superclass, it hides the superclass method rather than overriding it; the method called depends on the reference type, not the object type.

```
class Parent { static void hello() { System.out.println("Parent hello"); } }
class Child extends Parent { static void hello() { System.out.println("Child hello"); } }

Parent p = new Child();
p.hello();          // Parent hello   <- decided by REFERENCE type (hiding)
Child  c = new Child();
c.hello();          // Child hello
```

Part 2: Classes, Objects and Constructors

Q23. What is a Constructor in Java?

A constructor is a special method used to initialize objects. It has the same name as the class and no return type, and it is called automatically when an object is created.

Q24. What are the types of Constructors?

- **Default Constructor** - provided automatically by Java if no constructor is defined; it takes no arguments.
- **Parameterized Constructor** - accepts arguments to initialize fields with specific values.

Q25. What is Constructor Overloading?

Defining multiple constructors in a class with different parameter lists, allowing objects to be created in different ways.

> **Real-world: E-commerce Order**
> An order can be created empty (guest checkout), with a customer id, or with a customer id and coupon.

```
class Order {
    private final String customerId;
    private final String coupon;

    Order()                          { this("GUEST", "NONE"); }          // default-style
    Order(String customerId)         { this(customerId, "NONE"); }       // overloaded
    Order(String customerId, String coupon) {                            // main constructor
        this.customerId = customerId;
        this.coupon = coupon;
    }
}
Order o1 = new Order();
Order o2 = new Order("C101");
Order o3 = new Order("C101", "DIWALI20");
```

Q26. Can a constructor be private in Java?

Yes. A private constructor prevents direct instantiation of a class from outside, which is commonly used in Singleton design patterns.

```
class AppConfig {
    private static final AppConfig INSTANCE = new AppConfig();
    private AppConfig() {}                        // nobody outside can call new AppConfig()
    public static AppConfig getInstance() { return INSTANCE; }
}
// AppConfig c = new AppConfig();   // COMPILE ERROR
```

Q27. What is Constructor Chaining?

Constructor chaining is the process of calling one constructor from another within the same class (using `this()`) or from the parent class (using `super()`).

```
class Vehicle {
    Vehicle(String plate) { System.out.println("Vehicle: " + plate); }
}
class Car extends Vehicle {
    Car()              { this("TN-00-0000", 4); }            // chaining within same class
    Car(String plate, int seats) {
        super(plate);                                        // chaining to parent
        System.out.println("Car seats: " + seats);
    }
}
new Car();   // Vehicle: TN-00-0000  then  Car seats: 4
```

Q28. What is the difference between this() and super() in constructors?

`this()` calls another constructor in the same class. `super()` calls a constructor in the parent class. Both must be the first statement in a constructor, so they cannot be used together in the same constructor.

Q29. Does Java call a default constructor if we define a parameterized constructor?

No. If any constructor is explicitly defined, Java does not provide a default no-argument constructor automatically.

```
class User {
    User(String name) {}
}
// User u = new User();   // COMPILE ERROR - no no-arg constructor exists
User u = new User("Mathesh");   // OK
```

Q30. What is the "this" keyword used for in Java?

`this` refers to the current instance of the class. It is used to distinguish instance variables from parameters with the same name, to call another constructor, or to pass the current object as an argument.

```
class Driver {
    String name;
    Driver(String name) { this.name = name; }      // this.name = field, name = parameter
    Driver self()       { return this; }           // return current object
    void register(Registry r) { r.add(this); }     // pass current object
}
```

Q31. What is the "super" keyword used for in Java?

`super` refers to the immediate parent class. It is used to call the parent class's constructor, access parent class fields, or call parent class methods that have been overridden.

```
class Ride { double fare(double km) { return km * 10; } }
class NightRide extends Ride {
    @Override double fare(double km) {
        return super.fare(km) * 1.5;      // reuse parent logic, add night surcharge
    }
}
```

Q32. Can a constructor be inherited in Java?

No, constructors are not inherited by subclasses, but a subclass constructor can call a superclass constructor using `super()`.

Q33. What happens if a class has no constructor at all?

The Java compiler automatically provides a default no-argument constructor.

> Object creation order (common interview trap): static blocks of parent -> static blocks of child -> instance blocks + constructor of parent -> instance blocks + constructor of child.

Part 3: Access Modifiers and Keywords

Q34. What are Access Modifiers in Java?

They control the visibility/accessibility of classes, fields, methods, and constructors. The four types are: private, default (package-private), protected, and public.

Q35. What is the difference between private, default, protected, and public access modifiers?

- `private` - accessible only within the same class.
- `default` (no modifier) - accessible within the same package.
- `protected` - accessible within the same package and by subclasses in other packages.
- `public` - accessible from anywhere.

Modifier	Same class	Same package	Subclass (other pkg)	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Real-world: E-commerce Product
`private` for price/stock, `protected` for helpers that child classes (like Electronics) may reuse, `public` for the API other services call.

```
public class Product {
    private double price;               // only Product
    int stock;                          // same package (default)
    protected String category;          // package + subclasses
    public String name;                 // everyone
}
```

Q36. What is the static keyword used for?

`static` means a member (field, method, or block) belongs to the class itself rather than to any specific instance. It is shared across all objects of the class.

Real-world: Netflix
Total number of registered users is shared across the whole app, not per user - perfect for a static counter.

```
class Subscriber {
    static int totalSubscribers = 0;          // one copy shared by all objects
    String name;

    Subscriber(String name) { this.name = name; totalSubscribers++; }
    static int getTotal()   { return totalSubscribers; }   // call without an object
}
new Subscriber("Asha"); new Subscriber("Ravi");
System.out.println(Subscriber.getTotal());    // 2
```

Q37. Can we access non-static variables inside a static method?

No, non-static (instance) variables cannot be accessed directly inside a static method, since static methods do not operate on a specific object instance.

```
class Demo {
    int x = 10;
    static void test() {
        // System.out.println(x);        // COMPILE ERROR - which object's x?
        Demo d = new Demo();
        System.out.println(d.x);         // OK - through an explicit object
    }
}
```

Q38. What is a static block, and when is it executed?

A static block is used to initialize static variables. It runs once, when the class is first loaded into memory, before the `main()` method executes.

```
class DatabaseConfig {
    static String url;
    static {
        url = "jdbc:mysql://localhost:3306/shop";     // runs once at class loading
        System.out.println("Config loaded");
    }
}
```

Q39. What is the final keyword used for?

`final` can be applied to variables (making them constants, unmodifiable after assignment), methods (preventing them from being overridden), and classes (preventing them from being subclassed/inherited).

```
final class PaymentGatewayKey {}                 // cannot be extended (like String)
class Tax {
    static final double GST = 0.18;              // constant
    final double calculate(double amt) { return amt * GST; }   // cannot be overridden
}
```

Q40. Can a final method be overloaded?

Yes, a final method can be overloaded but cannot be overridden.

Q41. What is a final variable, and can it be initialized later?

A final variable's value cannot be changed once assigned. It can be left uninitialized at declaration and assigned exactly once later, typically in a constructor (called a "blank final" variable).

```
class Invoice {
    private final String invoiceId;              // blank final
    Invoice(String id) { this.invoiceId = id; }   // assigned exactly once
}
final List<String> items = new ArrayList<>();
items.add("Phone");          // OK - object state can change
// items = new ArrayList<>();   // COMPILE ERROR - reference cannot change
```

Q42. What is the difference between final, finally, and finalize()?

- `final` is a keyword/modifier used to restrict modification, inheritance, or overriding.
- `finally` is a block used in exception handling that always executes after a try-catch block.
- `finalize()` is a method called by the garbage collector before an object is destroyed (now deprecated).

Part 4: Abstract Classes and Interfaces

Q43. What is an Abstract Class?

An abstract class is a class that cannot be instantiated on its own and may contain both abstract methods (without a body) and concrete methods (with a body). It is declared using the `abstract` keyword. You cannot create an instance/object directly from an abstract class.

> **Real-world: Netflix content**
> Netflix has `Movie` and `Series`. Both share title, rating and play(), but "duration" is calculated differently. A generic `Content` is never watched directly, so it is abstract.

```
abstract class Content {
    protected String title;
    protected double rating;

    Content(String title, double rating) { this.title = title; this.rating = rating; }

    abstract int totalDurationMinutes();                // each type decides
    void play() { System.out.println("Playing " + title); }   // shared behavior
}

class Movie extends Content {
    private int minutes;
    Movie(String t, double r, int minutes) { super(t, r); this.minutes = minutes; }
    int totalDurationMinutes() { return minutes; }
}

class Series extends Content {
    private int episodes, minutesPerEpisode;
    Series(String t, double r, int e, int m) { super(t, r); episodes = e; minutesPerEpisode = m; }
    int totalDurationMinutes() { return episodes * minutesPerEpisode; }
}
// Content c = new Content(...);   // COMPILE ERROR - abstract
Content c = new Series("Stranger Things", 8.7, 8, 50);
System.out.println(c.totalDurationMinutes());           // 400
```

Q44. What is an Abstract Method?

An abstract method is a method declared without an implementation (no body); it must be implemented by any concrete subclass.

Q45. Can an abstract class have a constructor?

Yes, an abstract class can have a constructor, which is called when a subclass object is instantiated (via `super()`). (See `Content` above: `Movie` calls `super(t, r)`.)

Q46. Can an abstract class have no abstract methods?

Yes, a class can be declared abstract even if it has no abstract methods, purely to prevent direct instantiation.

Q47. What is an Interface in Java?

An interface is a reference type, similar to a class, that can contain only abstract methods (until Java 8), constants, default methods, and static methods. It represents a contract that implementing classes must fulfill.

```
interface Streamable {
    int MAX_QUALITY = 4;              // implicitly public static final
    void stream(String quality);      // implicitly public abstract
}
class Movie implements Streamable {
    public void stream(String quality) { System.out.println("Streaming in " + quality); }
}
```

Q48. What is the difference between an Abstract Class and an Interface?

An abstract class can have both abstract and concrete methods, constructors, and instance variables, and supports single inheritance. An interface (traditionally) only had abstract methods and constants, supports multiple inheritance, and (since Java 8) can have default and static methods, but still cannot have constructors or instance state.

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract, default, static, private (Java 9)
Fields	Any (instance variables allowed)	Only public static final constants
Constructors	Yes	No
Inheritance	Extend only ONE class	Implement MANY interfaces
Access modifiers	Any	Methods are public
Use when	Closely related classes share state/code	Unrelated classes share a capability

Q49. Why use an Interface instead of an Abstract Class?

Interfaces allow a class to implement multiple interfaces, achieving a form of multiple inheritance, and are used to define a contract that unrelated classes can share.

Real-world: Uber
A `Driver` can also be a `Rider`; a `Car` and a `Bike` can both be `Chargeable` (EV) without sharing a parent. Capabilities = interfaces; identity = abstract class.

```
interface Payable   { void pay(double amt); }
interface Trackable { String currentLocation(); }

class Ride implements Payable, Trackable {         // multiple interfaces: OK
    public void pay(double amt)       { System.out.println("Paid " + amt); }
    public String currentLocation()   { return "11.0168 N, 76.9558 E"; }
}
```

Q50. Can an Interface extend another Interface?

Yes, an interface can extend one or more other interfaces using the `extends` keyword.

```
interface Playable { void play(); }
interface Downloadable { void download(); }
interface OfflineVideo extends Playable, Downloadable {}   // interface extends many
```

Q51. Can a class implement multiple interfaces?

Yes, a class can implement multiple interfaces using a comma-separated list.

Q52. What are Default Methods in Interfaces (Java 8)?

Default methods are methods in an interface that have a body/implementation, introduced to allow interfaces to evolve without breaking existing implementing classes. The implementing class can choose to use the default implementation or override it with its own logic.

Real-world: Netflix
Netflix adds a new feature "skipIntro()" to the `Playable` interface. Thousands of existing classes should not break, so it is added as a default method.

```
interface Playable {
    void play();
    default void skipIntro() { System.out.println("Skipping intro (default)"); }
}
class Movie implements Playable {
    public void play() { System.out.println("Movie playing"); }
}
class Series implements Playable {
```

```
    public void play() { System.out.println("Series playing"); }
    @Override public void skipIntro() { System.out.println("Skipping 90-sec intro"); }
}
new Movie().skipIntro();    // default
new Series().skipIntro();   // overridden
```

Difference between a Concrete method in an Abstract class and a Default method in an Interface

Aspect	Concrete method (Abstract class)	Default method (Interface)
Access modifier	public, protected, private, default	Always public
Instance state	Can read/modify object fields	Cannot access instance fields
Inheritance	Class supports only single inheritance	Interface supports multiple inheritance
Constructors	Abstract class has constructors	Interface has no constructors

Q53. What are Static Methods in Interfaces?

Introduced in Java 8, these are utility methods that belong to the interface itself and can be called without an implementing object, similar to static methods in classes. They are commonly used to provide utility or helper methods directly related to the interface contract.

```
interface Validator {
    boolean validate(String input);
    static boolean isEmpty(String s) { return s == null || s.isBlank(); }   // utility
}
Validator.isEmpty("  ");     // true - called on the interface, not on an object
```

Q54. What happens if a class implements two interfaces with the same default method?

The implementing class must explicitly override the method to resolve the conflict, otherwise a compile-time error occurs.

```
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }

class C implements A, B {
    @Override public void hello() {
        A.super.hello();                // explicitly choose A's version (or B.super.hello())
    }
}
```

Q55. What is a Marker Interface?

A marker interface is an interface with no methods or fields (e.g., `Serializable`, `Cloneable`). It is used to signal metadata about a class to the JVM or other code.

Q56. Can an interface have private methods?

Yes, since Java 9, interfaces can have private methods, primarily used to share code between default methods within the same interface.

```
interface Logger {
    default void info(String m)  { print("INFO", m); }
    default void error(String m) { print("ERROR", m); }
    private void print(String level, String m) { System.out.println("[" + level + "] " + m); }
}
```

Q57. Can an interface have fields?

Yes, but they are implicitly `public`, `static`, and `final` (constants).

Part 5: Polymorphism, Binding and Object Behavior

Q58. What is Dynamic Method Dispatch?

It is the mechanism by which a call to an overridden method is resolved at runtime, based on the actual object type rather than the reference type. This is the basis of runtime polymorphism in Java.

> **Real-world: Uber**
> The app holds a `List<Ride>`. For each ride it just calls `calculateFare()`. At runtime Java picks AutoRide's, PremiumRide's or BikeRide's version.

```
List<Ride> todayRides = List.of(new AutoRide(), new PremiumRide(), new Ride());
for (Ride r : todayRides) {
    System.out.println(r.calculateFare(10));    // 150.0, 350.0, 100.0 - decided at runtime
}
```

Q59. What is the difference between Static Binding and Dynamic Binding?

Static binding (early binding) is resolved at compile time, used for overloaded methods, private methods, and static methods. Dynamic binding (late binding) is resolved at runtime, used for overridden methods.

Q60. What is Upcasting and Downcasting in Java?

Upcasting is casting a subclass reference to a superclass type, done implicitly and safely. Downcasting is casting a superclass reference back to a subclass type, done explicitly, and can throw a `ClassCastException` if the object isn't actually of that subclass type.

Q61. What is the instanceof operator used for?

It checks whether an object is an instance of a specific class or interface, commonly used before downcasting to avoid `ClassCastException`.

```
Content c = new Movie("Leo", 7.4, 160);       // upcasting (implicit)

if (c instanceof Movie) {                      // safe check
    Movie m = (Movie) c;                       // downcasting (explicit)
    System.out.println("Movie: " + m.totalDurationMinutes());
}
Series s = (Series) c;                         // RUNTIME ERROR: ClassCastException

// Java 16+ pattern matching - check and cast in one step
if (c instanceof Movie m2) { System.out.println(m2.totalDurationMinutes()); }
```

Q62. Can we override a private method?

No, private methods are not visible to subclasses, so they cannot be overridden. A subclass defining a method with the same signature simply creates a new, unrelated method.

Q63. Can constructors be overridden?

No, constructors are not inherited and therefore cannot be overridden. They can, however, be overloaded.

Q64. What is Covariant Return Type?

Since Java 5, an overriding method can return a subtype of the return type declared in the overridden method in the parent class.

```
class Vehicle { }
class Car extends Vehicle { }

class Garage {
    Vehicle getVehicle() { return new Vehicle(); }
}
class CarGarage extends Garage {
    @Override Car getVehicle() { return new Car(); }     // Car is a subtype of Vehicle: OK
```

```
}
```

Q65. What are the rules for Method Overriding in Java?

- The method name and parameters must be exactly the same.
- The return type must be the same or a covariant type.
- Access modifier cannot be more restrictive than the parent method.
- The overriding method cannot throw broader checked exceptions than the parent method.
- The method must not be static, private, or final in the parent class.

Part 6: Object Class Methods

Q66. What is the Object class in Java?

`Object` is the root class of the Java class hierarchy. Every class in Java implicitly extends `Object` if it does not explicitly extend another class.

Q67. What is the difference between == and .equals()?

`==` compares references (memory addresses) for objects, or actual values for primitives. `.equals()` compares the logical content/state of objects, and its behavior depends on how it is overridden in the class.

Q68. What is the purpose of overriding equals() and hashCode() together?

The general contract requires that if two objects are equal according to `equals()`, they must return the same `hashCode()`. Overriding one without the other can break the behavior of hash-based collections like HashMap and HashSet.

> **Real-world: E-commerce**
> Two `Product` objects with the same SKU are the "same product". If the cart is a `HashSet<Product>`, it must not hold duplicates of that SKU.

```
class Product {
    private final String sku;
    private final String name;
    Product(String sku, String name) { this.sku = sku; this.name = name; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Product p = (Product) o;
        return sku.equals(p.sku);                    // identity = SKU
    }
    @Override public int hashCode() { return java.util.Objects.hash(sku); }
}
Set<Product> cart = new HashSet<>();
cart.add(new Product("SKU1", "Phone"));
cart.add(new Product("SKU1", "Phone"));
System.out.println(cart.size());     // 1 (with both overridden)  | 2 if hashCode is missing
```

Q69. What happens if we override equals() but not hashCode()?

Two logically equal objects may end up with different hash codes, causing inconsistent behavior in hash-based collections (e.g., duplicate entries in a HashSet).

Q70. What is the toString() method used for?

It returns a string representation of an object. By default, it returns the class name plus a hash code, but it is commonly overridden to provide meaningful output.

```
class Ride {
    String pickup = "A", drop = "B";
    @Override public String toString() { return "Ride[" + pickup + " -> " + drop + "]"; }
}
System.out.println(new Ride());      // Ride[A -> B]   (instead of Ride@1b6d3586)
```

Q71. What is the clone() method used for?

It creates and returns a copy of an object. The class must implement the `Cloneable` marker interface, or a `CloneNotSupportedException` is thrown.

Q72. What is the difference between Shallow Copy and Deep Copy?

A shallow copy copies an object's fields as-is, so reference fields still point to the same objects as the original. A deep copy recursively copies all objects referenced by the fields as well, creating fully independent copies.

Real-world: E-commerce cart
A "Save for later" feature copies a cart. With a shallow copy, removing an item from the copy also removes it from the original.

```
class Cart {
    List<String> items = new ArrayList<>();

    Cart shallowCopy() { Cart c = new Cart(); c.items = this.items; return c; }          // same list!
    Cart deepCopy()    { Cart c = new Cart(); c.items = new ArrayList<>(this.items); return c; }
}
Cart a = new Cart(); a.items.add("Phone");
Cart s = a.shallowCopy(); s.items.add("Case");
System.out.println(a.items);      // [Phone, Case]  <- original affected (shallow)
Cart d = a.deepCopy();   d.items.add("Charger");
System.out.println(a.items);      // [Phone, Case]  <- original NOT affected (deep)
```

Q73. What is the finalize() method?

It is a method called by the garbage collector before reclaiming an object's memory, allowing cleanup. It is deprecated since Java 9 in favor of try-with-resources and the Cleaner API.

Q74. What is the getClass() method used for?

It returns the runtime class of an object, useful for reflection and type checking.

Part 7: Exception Handling (OOP Context)

Q75. What is Exception Handling in Java?

It is a mechanism to handle runtime errors gracefully using `try`, `catch`, `finally`, `throw`, and `throws`, preventing abnormal program termination.

Q76. What is the difference between Checked and Unchecked Exceptions?

Checked exceptions are checked at compile time and must be either caught or declared using `throws` (e.g., `IOException`). Unchecked exceptions extend `RuntimeException` and are not checked at compile time (e.g., `NullPointerException`).

Q77. What is the difference between throw and throws?

`throw` is used to explicitly raise an exception within a method. `throws` is used in a method signature to declare that a method might throw certain exceptions.

Q78. What is a custom exception, and how do you create one?

A custom exception is a user-defined exception class, typically created by extending `Exception` (checked) or `RuntimeException` (unchecked), used to represent application-specific error conditions.

> **Real-world: Uber payments**
> When a rider's wallet cannot cover the fare, the system raises a domain-specific exception that the UI can turn into "Please add money".

```
class InsufficientBalanceException extends Exception {          // checked
    private final double shortBy;
    public InsufficientBalanceException(double shortBy) {
        super("Insufficient balance. Short by " + shortBy);
        this.shortBy = shortBy;
    }
    public double getShortBy() { return shortBy; }
}

class RiderWallet {
    private double balance = 100;
    void payFare(double fare) throws InsufficientBalanceException {    // throws = declares
        if (fare > balance) throw new InsufficientBalanceException(fare - balance);  // throw = raises
        balance -= fare;
    }
}
try {
    new RiderWallet().payFare(250);
} catch (InsufficientBalanceException e) {
    System.out.println(e.getMessage());         // Insufficient balance. Short by 150.0
}
```

Q79. What is the finally block used for?

The `finally` block contains code that always executes after a try-catch block, regardless of whether an exception occurred, typically used for cleanup (e.g., closing resources).

Q80. Can we have a try block without a catch block?

Yes, as long as it is followed by a `finally` block, or used as a try-with-resources statement.

Q81. What is try-with-resources?

Introduced in Java 7, it automatically closes resources (that implement `AutoCloseable`) at the end of the try block, without needing an explicit `finally` block.

```
// Old way
Connection con = null;
try { con = DriverManager.getConnection(url); /* use */ }
catch (SQLException e) { e.printStackTrace(); }
```

```
finally { if (con != null) try { con.close(); } catch (SQLException ignored) {} }

// try-with-resources: con.close() is called automatically, even on exception
try (Connection c = DriverManager.getConnection(url);
     Statement st = c.createStatement()) {
    st.executeQuery("SELECT * FROM orders");
} catch (SQLException e) {
    e.printStackTrace();
}
```

Part 8: Java-Specific OOP Concepts

Q82. What is the difference between an Interface and an Abstract Class in terms of "IS-A" relationships?

Both represent an "IS-A" relationship, but interfaces are typically used to define capability ("CAN-DO"), while abstract classes represent a more specific shared identity among closely related classes.

Q83. What is Composition in OOP?

Composition is a design principle where a class contains references to other objects as fields, modeling a "HAS-A" relationship, rather than inheriting behavior.

> **Real-world: E-commerce**
> An `Order` HAS-A list of `OrderItem`s. If the order is deleted, its items have no meaning - they die with it.

```
class OrderItem {
    final String sku; final int qty; final double price;
    OrderItem(String sku, int qty, double price) { this.sku = sku; this.qty = qty; this.price = price; }
    double total() { return qty * price; }
}
class Order {
    private final List<OrderItem> items = new ArrayList<>();   // created and owned by Order
    void addItem(String sku, int qty, double price) { items.add(new OrderItem(sku, qty, price)); }
    double grandTotal() { return items.stream().mapToDouble(OrderItem::total).sum(); }
}
```

Q84. What is the difference between Inheritance and Composition?

Inheritance models an "IS-A" relationship and reuses code by extending a class. Composition models a "HAS-A" relationship and reuses code by including instances of other classes as fields. Composition is often preferred for greater flexibility and to avoid tight coupling.

Q85. Why is Composition often preferred over Inheritance?

Composition provides more flexibility, avoids issues like the fragile base class problem, allows behavior to be changed at runtime, and avoids exposing unnecessary parent class methods.

> **Real-world: Uber**
> Instead of `class UberXRide extends PremiumRide extends EVRide...`, a `Ride` HAS-A `PricingStrategy` and HAS-A `VehicleType`. You can swap the strategy at runtime (e.g., surge pricing) without touching the class hierarchy.

```
interface PricingStrategy { double price(double km); }

class Ride {
    private PricingStrategy pricing;                    // HAS-A (composition)
    Ride(PricingStrategy pricing) { this.pricing = pricing; }
    void setPricing(PricingStrategy p) { this.pricing = p; }   // change behavior at runtime
    double fare(double km) { return pricing.price(km); }
}
Ride ride = new Ride(km -> km * 12);
ride.setPricing(km -> km * 12 * 2.0);                  // surge kicks in
```

Q86. What is Aggregation?

Aggregation is a specialized form of association representing a "HAS-A" relationship where the contained object can exist independently of the container object (a weaker form of composition).

Q87. What is the difference between Aggregation and Composition?

In composition, the contained object's lifecycle is tightly bound to the container (it cannot exist independently). In aggregation, the contained object can exist independently of the container.

Real-world: Netflix
A `Playlist` aggregates `Movie`s - delete the playlist and the movies are still in the catalog. An `Order` composes `OrderItem`s - delete the order and the items vanish.

```
class Playlist {                                          // AGGREGATION
    private final List<Movie> movies = new ArrayList<>();
    void add(Movie m) { movies.add(m); }                  // Movie created OUTSIDE, lives on its own
}
class Order2 {                                            // COMPOSITION
    private final List<OrderItem> items = new ArrayList<>();
    void addItem(String sku, int q, double p) { items.add(new OrderItem(sku, q, p)); }  // created INSIDE
}
```

Q88. What is Association in OOP?

Association is a general relationship between two separate classes/objects that are connected but do not depend on each other's existence.

Real-world: Uber
A `Driver` and a `Rider` are associated through a ride, but neither owns the other.

Relationship	Meaning	Strength	Example
Association	Uses / knows	Weakest	Driver - Rider
Aggregation	HAS-A, independent lifecycle	Medium	Playlist - Movie
Composition	HAS-A, dependent lifecycle	Strongest	Order - OrderItem
Inheritance	IS-A	Class-level	Car IS-A Vehicle

Q89. What is an Anonymous Class in Java?

An anonymous class is a class without a name, defined and instantiated in a single expression, typically used to provide a one-time implementation of an interface or abstract class.

```
interface OnComplete { void done(String msg); }

OnComplete cb = new OnComplete() {                // anonymous class
    @Override public void done(String msg) { System.out.println("Callback: " + msg); }
};
cb.done("Payment successful");
```

Q90. What is an Inner Class (Nested Class)?

A class defined within another class. Types include static nested classes, non-static (inner) classes, local classes, and anonymous classes.

Q91. What is the difference between a Static Nested Class and a Non-static Inner Class?

A static nested class does not have access to the instance members of the outer class and can be instantiated without an outer class object. A non-static inner class is tied to an instance of the outer class and can access its instance members directly.

```
class Netflix {
    private String region = "IN";

    static class Config {                         // static nested
        String version = "v1";
    }
    class Session {                               // inner (non-static)
        String info() { return "Session in " + region; }   // can read outer's fields
    }
}
Netflix.Config cfg = new Netflix.Config();        // no outer object needed
Netflix.Session s  = new Netflix().new Session();   // needs an outer object
```

Q92. What is a Local Class in Java?

A class defined within a method body, visible only within that method's scope.

Q93. What is a Singleton class, and how do you implement it?

A Singleton class ensures that only one instance of the class is created throughout the application. It is implemented by making the constructor private, providing a static method to return the single instance, and holding that instance in a private static field.

> **Real-world: Netflix / E-commerce**
> One `CatalogService`, one `ConnectionPool`, one `AppConfig` for the whole app. Creating many would waste memory or cause inconsistent state.

Q94. What are the ways to implement a Singleton pattern in Java?

Eager initialization, Lazy initialization, Thread-safe Singleton (with synchronized), Double-checked locking, Bill Pugh Singleton (static inner class), and Enum-based Singleton.

```
// 1. Eager - created at class loading (simple, thread-safe)
class EagerSingleton {
    private static final EagerSingleton INSTANCE = new EagerSingleton();
    private EagerSingleton() {}
    public static EagerSingleton getInstance() { return INSTANCE; }
}

// 2. Lazy - NOT thread-safe (two threads can create two objects)
class LazySingleton {
    private static LazySingleton instance;
    private LazySingleton() {}
    public static LazySingleton getInstance() {
        if (instance == null) instance = new LazySingleton();
        return instance;
    }
}

// 3. Synchronized - safe but slow (locks on every call)
public static synchronized LazySingleton getInstance() { ... }

// 4. Double-checked locking - safe and fast (volatile is mandatory)
class DclSingleton {
    private static volatile DclSingleton instance;
    private DclSingleton() {}
    public static DclSingleton getInstance() {
        if (instance == null) {
            synchronized (DclSingleton.class) {
                if (instance == null) instance = new DclSingleton();
            }
        }
        return instance;
    }
}

// 5. Bill Pugh - lazy + thread-safe, no synchronization (class loader guarantees it)
class BillPugh {
    private BillPugh() {}
    private static class Holder { static final BillPugh INSTANCE = new BillPugh(); }
    public static BillPugh getInstance() { return Holder.INSTANCE; }
}

// 6. Enum - best (Effective Java): safe from reflection and serialization attacks
enum AppConfigSingleton {
    INSTANCE;
    public String dbUrl = "jdbc:mysql://localhost/shop";
}
// usage: AppConfigSingleton.INSTANCE.dbUrl
```

Q95. Why is Enum-based Singleton considered the best approach?

It is inherently thread-safe, protects against reflection and serialization attacks, and requires minimal code.

Q96. What is Method Chaining in OOP?

Method chaining is a technique where multiple methods are called on the same object in a single statement, achieved by having each method return the current object (`this`).

```
class QueryBuilder {
    private final StringBuilder q = new StringBuilder();
    QueryBuilder select(String cols) { q.append("SELECT ").append(cols); return this; }
    QueryBuilder from(String table)  { q.append(" FROM ").append(table); return this; }
    QueryBuilder where(String cond)  { q.append(" WHERE ").append(cond); return this; }
    String build() { return q.toString(); }
}
String sql = new QueryBuilder().select("*").from("orders").where("status='PAID'").build();
```

Part 9: Generics and Collections

Q97. What are Generics in Java?

Generics allow classes, interfaces, and methods to operate on types specified as parameters, providing type safety at compile time and eliminating the need for explicit casting.

> **Real-world: Spring Boot / E-commerce**
> A single `Repository<T, ID>` works for `Product`, `Order`, `User`. This is exactly the idea behind Spring Data's `JpaRepository<T, ID>`.

```
interface Repository<T, ID> {
    void save(T entity);
    T findById(ID id);
}
class ProductRepository implements Repository<Product, String> {
    private final Map<String, Product> db = new HashMap<>();
    public void save(Product p)     { db.put(p.getSku(), p); }
    public Product findById(String sku) { return db.get(sku); }   // no casting needed
}
```

Q98. Why use Generics instead of Object type?

Generics provide compile-time type checking, eliminate the need for casting, and reduce the risk of `ClassCastException` at runtime.

```
List raw = new ArrayList();            // old style
raw.add("Phone"); raw.add(42);         // compiles!
String s = (String) raw.get(1);        // RUNTIME ClassCastException

List<String> safe = new ArrayList<>();
safe.add("Phone");
// safe.add(42);                       // COMPILE ERROR - caught early
```

Q99. What is Type Erasure in Java Generics?

Type erasure is the process by which the compiler removes all generic type information after compilation, replacing type parameters with their bounds or `Object`, ensuring backward compatibility with older Java versions.

> Consequence: at runtime `List<String>` and `List<Integer>` are both just `List`. So `new T()`, `T.class` and `instanceof List<String>` are not allowed.

Q100. What is the difference between List, Set, and Map in Java Collections?

- `List` - an ordered collection that allows duplicate elements.
- `Set` - a collection that does not allow duplicate elements.
- `Map` - a collection of key-value pairs, where each key is unique.

```
List<String> watchHistory = new ArrayList<>(List.of("Leo", "Jailer", "Leo"));   // duplicates OK, ordered
Set<String> uniqueGenres  = new HashSet<>(List.of("Action", "Drama", "Action")); // [Action, Drama]
Map<String, Integer> ratingByMovie = new HashMap<>();
ratingByMovie.put("Leo", 8);                    // key -> value, key unique
```

Q101. What is the difference between ArrayList and LinkedList?

ArrayList uses a dynamic array internally, offering fast random access but slower insertions/deletions in the middle. LinkedList uses a doubly linked list, offering faster insertions/deletions but slower random access.

Operation	ArrayList	LinkedList
get(index)	O(1)	O(n)
add at end	O(1) amortized	O(1)
insert/remove in middle	O(n)	O(1) once node is found (O(n) to find)

Operation	ArrayList	LinkedList
Memory	Less (array)	More (node + 2 pointers)
Best for	Reading, iterating	Frequent insert/remove at ends

Q102. What is the difference between HashMap and TreeMap?

HashMap does not guarantee any order of keys and offers average O(1) time complexity for get/put. TreeMap stores keys in sorted order and offers O(log n) time complexity, as it is backed by a Red-Black tree.

Q103. What is the difference between Comparable and Comparator?

Comparable is implemented by the class itself to define a natural ordering (via `compareTo()`). Comparator is a separate class used to define custom ordering (via `compare()`), allowing multiple sort orders for the same class.

Real-world: Netflix
Default order of movies = by rating (Comparable). A user can also sort by release year or title (Comparator).

```
class Film implements Comparable<Film> {
    String title; double rating; int year;
    Film(String t, double r, int y) { title = t; rating = r; year = y; }
    @Override public int compareTo(Film o) { return Double.compare(o.rating, this.rating); } // high first
    @Override public String toString() { return title; }
}
List<Film> films = new ArrayList<>(List.of(new Film("A", 7.5, 2020), new Film("B", 8.9, 2018)));
Collections.sort(films);                                    // natural: by rating -> [B, A]
films.sort(Comparator.comparingInt((Film f) -> f.year));    // custom: by year   -> [B, A]
films.sort(Comparator.comparing((Film f) -> f.title).reversed()); // custom: by title desc
```

Part 10: SOLID Principles and Design Patterns

Q104. What are SOLID principles in OOP?

- **S**ingle Responsibility Principle - a class should have only one reason to change.
- **O**pen/Closed Principle - classes should be open for extension but closed for modification.
- **L**iskov Substitution Principle - subclasses should be substitutable for their base classes without breaking behavior.
- **I**nterface Segregation Principle - clients should not be forced to depend on interfaces they don't use.
- **D**ependency Inversion Principle - depend on abstractions, not concrete implementations.

Q105. What is the Single Responsibility Principle?

A class should have only one reason to change, meaning it should only have one job or responsibility.

```
// BAD: one class does everything
class OrderServiceBad { void placeOrder() {} void sendEmail() {} void saveToDb() {} void
    generateInvoicePdf() {} }

// GOOD: each class has one job
class OrderService   { void placeOrder(Order o) {} }
class EmailService   { void sendConfirmation(Order o) {} }
class OrderRepository{ void save(Order o) {} }
class InvoiceService { byte[] generatePdf(Order o) { return new byte[0]; } }
```

Q106. What is the Open/Closed Principle?

Software entities (classes, modules, functions) should be open for extension but closed for modification, typically achieved through abstraction and polymorphism.

> **Real-world: E-commerce discounts**
> Adding a "Festival discount" must not require editing existing, tested code - just add a new class.

```
interface Discount { double apply(double amount); }
class NoDiscount       implements Discount { public double apply(double a) { return a; } }
class FestivalDiscount implements Discount { public double apply(double a) { return a * 0.80; } }
class StudentDiscount  implements Discount { public double apply(double a) { return a * 0.90; } }  //
    // NEW: no edits elsewhere

class BillingService {
    double finalAmount(double amount, Discount d) { return d.apply(amount); }   // never changes
}
```

Q107. What is the Liskov Substitution Principle?

Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

```
// VIOLATION: a "Bird" that cannot fly breaks code expecting every Bird to fly
class Bird { void fly() {} }
class Penguin extends Bird { @Override void fly() { throw new UnsupportedOperationException(); } }

// FIX: model capabilities separately
class Bird2 {}
interface Flyable { void fly(); }
class Sparrow extends Bird2 implements Flyable { public void fly() {} }
class Penguin2 extends Bird2 {}
```

Q108. What is the Interface Segregation Principle?

Clients should not be forced to depend on methods they do not use; it's better to have several small, specific interfaces than one large, general-purpose interface.

```
// BAD: Bike driver is forced to implement carryLuggage()
interface DriverActions { void drive(); void carryLuggage(); void playMusic(); }

// GOOD: small interfaces
interface Drivable { void drive(); }
```

```
interface LuggageCarrier { void carryLuggage(); }
class BikeDriver implements Drivable {}                          // only what it needs
class CabDriver  implements Drivable, LuggageCarrier {}
```

Q109. What is the Dependency Inversion Principle?

High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces), not concrete implementations.

Real-world: Spring Boot
`OrderService` (high-level) depends on `PaymentGateway` (interface), not on `RazorpayClient` (low-level). Spring injects the implementation - that is Dependency Injection, the technique that implements DIP.

```
interface PaymentGateway { void charge(double amt); }
class RazorpayGateway implements PaymentGateway { public void charge(double a) { /* Razorpay API */ } }
class StripeGateway   implements PaymentGateway { public void charge(double a) { /* Stripe API */ } }

class OrderService {
    private final PaymentGateway gateway;                         // abstraction
    OrderService(PaymentGateway gateway) { this.gateway = gateway; }   // constructor injection
    void checkout(double amt) { gateway.charge(amt); }
}
new OrderService(new RazorpayGateway()).checkout(999);
new OrderService(new StripeGateway()).checkout(999);              // swap without editing OrderService
```

Q110. What is Tight Coupling vs. Loose Coupling?

Tight coupling means classes are heavily dependent on each other's concrete implementations, making changes difficult. Loose coupling means classes interact through abstractions (interfaces), making the system more flexible and easier to maintain.

```
// TIGHT: OrderService is welded to Razorpay
class TightOrderService {
    private RazorpayGateway g = new RazorpayGateway();
    void checkout(double amt) { g.charge(amt); }
}
// LOOSE: see OrderService above - depends on PaymentGateway interface only
```

Q111. What is Cohesion in OOP?

Cohesion refers to how closely related and focused the responsibilities of a single class or module are. High cohesion (a class doing one well-defined job) is generally desirable.

Golden rule: aim for HIGH cohesion and LOW coupling.

Q112. What is a Design Pattern?

A design pattern is a general, reusable solution to a commonly occurring problem in software design.

Q113. What are the types of Design Patterns?

- **Creational** (e.g., Singleton, Factory, Builder) - how objects are created.
- **Structural** (e.g., Adapter, Decorator, Facade) - how objects are composed.
- **Behavioral** (e.g., Observer, Strategy, Command) - how objects communicate.

Q114. What is the Factory Design Pattern?

It defines an interface for creating objects but lets subclasses (or a factory method) decide which class to instantiate, decoupling object creation from usage.

Real-world: Uber
When a rider picks "Auto", "Mini" or "Premium", the app asks a factory for the right `Ride` object instead of using `new` everywhere.

```
class RideFactory {
    static Ride create(String type) {
        switch (type.toUpperCase()) {
            case "AUTO":    return new AutoRide();
            case "PREMIUM": return new PremiumRide();
            case "MINI":    return new Ride();
            default: throw new IllegalArgumentException("Unknown ride type: " + type);
        }
    }
}
Ride r = RideFactory.create("premium");      // caller never says "new PremiumRide()"
```

Q115. What is the Builder Design Pattern?

It is used to construct complex objects step by step, separating the construction process from the final representation, often used when a class has many optional parameters.

Real-world: E-commerce
An order has many optional parts (coupon, gift wrap, note, express delivery). A telescoping constructor with 8 parameters is unreadable - use a builder.

```
class Shipment {
    private final String address; private final boolean express, giftWrap; private final String note;
    private Shipment(Builder b) { address = b.address; express = b.express; giftWrap = b.giftWrap; note =
        b.note; }

    static class Builder {
        private final String address;                // mandatory
        private boolean express, giftWrap; private String note = "";
        Builder(String address) { this.address = address; }
        Builder express(boolean v)  { this.express = v;  return this; }
        Builder giftWrap(boolean v) { this.giftWrap = v; return this; }
        Builder note(String n)      { this.note = n;     return this; }
        Shipment build() { return new Shipment(this); }
    }
}
Shipment s = new Shipment.Builder("12, Avinashi Rd, Coimbatore").express(true).giftWrap(true).build();
```

Q116. What is the Observer Design Pattern?

It defines a one-to-many dependency between objects, so that when one object (subject) changes state, all its dependents (observers) are notified automatically.

Real-world: Netflix / Uber
When a new episode drops, every subscriber gets a notification. When a driver arrives, the rider's app, SMS service and trip-tracker all update.

```
interface Subscriber { void update(String message); }

class Show {                                                  // Subject
    private final List<Subscriber> subscribers = new ArrayList<>();
    void subscribe(Subscriber s)   { subscribers.add(s); }
    void unsubscribe(Subscriber s) { subscribers.remove(s); }
    void releaseEpisode(String ep) { subscribers.forEach(s -> s.update("New episode: " + ep)); }
}
class MobileApp implements Subscriber { public void update(String m) { System.out.println("Push: " + m);
    } }
class EmailAlert implements Subscriber { public void update(String m) { System.out.println("Mail: " + m);
    } }

Show show = new Show();
show.subscribe(new MobileApp()); show.subscribe(new EmailAlert());
show.releaseEpisode("S2E1");        // Push: New episode: S2E1   /   Mail: New episode: S2E1
```

Q117. What is the Strategy Design Pattern?

It defines a family of interchangeable algorithms, encapsulates each one, and allows the algorithm to be selected/changed at runtime without altering the client code.

Real-world: Uber surge pricing
Normal, night, rain-surge and event-surge are different pricing algorithms chosen at runtime. (Netflix recommendation algorithms - "trending", "because you watched" - work the same way.)

```
interface FareStrategy { double fare(double km); }
class NormalFare implements FareStrategy { public double fare(double km) { return km * 12; } }
class SurgeFare  implements FareStrategy {
    private final double multiplier;
    SurgeFare(double m) { this.multiplier = m; }
    public double fare(double km) { return km * 12 * multiplier; }
}
class FareCalculator {
    private FareStrategy strategy = new NormalFare();
    void setStrategy(FareStrategy s) { this.strategy = s; }
    double calculate(double km) { return strategy.fare(km); }
}
FareCalculator calc = new FareCalculator();
System.out.println(calc.calculate(10));       // 120.0
calc.setStrategy(new SurgeFare(1.8));         // rain starts
System.out.println(calc.calculate(10));       // 216.0
```

Q118. What is the Decorator Design Pattern?

It allows behavior to be added to an individual object dynamically, without affecting other objects of the same class, by wrapping the object in a decorator class.

Real-world: E-commerce
A product can be wrapped with gift-wrap, extended warranty, express delivery - each adds cost/description, in any combination.

```
interface Item { double cost(); String desc(); }
class Phone implements Item { public double cost() { return 20000; } public String desc() { return
    "Phone"; } }

abstract class ItemDecorator implements Item {
    protected final Item item;
    ItemDecorator(Item item) { this.item = item; }
}
class GiftWrap extends ItemDecorator {
    GiftWrap(Item i) { super(i); }
    public double cost() { return item.cost() + 100; }
    public String desc() { return item.desc() + " + Gift wrap"; }
}
class Warranty extends ItemDecorator {
    Warranty(Item i) { super(i); }
    public double cost() { return item.cost() + 1500; }
    public String desc() { return item.desc() + " + 1yr warranty"; }
}
Item order = new Warranty(new GiftWrap(new Phone()));
System.out.println(order.desc() + " = " + order.cost());   // Phone + Gift wrap + 1yr warranty = 21600.0
```

Q119. What is the Adapter Design Pattern?

It allows incompatible interfaces to work together by converting the interface of one class into another interface that clients expect.

Real-world: E-commerce payments
Your app expects `PaymentGateway.charge()`, but a third-party SDK exposes `makePayment(amountInPaise)`. An adapter bridges the two.

```
class LegacyPaytm { void makePayment(long amountInPaise) { System.out.println("Paytm: " + amountInPaise);
    } }

class PaytmAdapter implements PaymentGateway {                // PaymentGateway: our own interface
    private final LegacyPaytm paytm = new LegacyPaytm();
    @Override public void charge(double amountInRupees) {
        paytm.makePayment((long) (amountInRupees * 100));       // convert to what SDK expects
    }
}
```

```
PaymentGateway g = new PaytmAdapter();
g.charge(499.50);        // Paytm: 49950
```

Q120. What is the Difference between Factory Pattern and Abstract Factory Pattern?

The Factory Pattern creates objects of a single type through one method. The Abstract Factory Pattern provides an interface for creating families of related objects without specifying their concrete classes.

Real-world: E-commerce regions
India store uses UPI + GST invoice; US store uses Card + Sales-tax invoice. Each region factory produces a consistent family of objects.

```
interface TaxCalculator { double tax(double amt); }
interface InvoiceFormat { String format(double total); }

interface RegionFactory {                    // Abstract Factory
    PaymentMethod createPayment();
    TaxCalculator createTax();
}
class IndiaFactory implements RegionFactory {
    public PaymentMethod createPayment() { return new UpiPayment(); }
    public TaxCalculator createTax()     { return amt -> amt * 0.18; }     // GST
}
class UsFactory implements RegionFactory {
    public PaymentMethod createPayment() { return new CardPayment(); }
    public TaxCalculator createTax()     { return amt -> amt * 0.07; }     // sales tax
}
RegionFactory f = new IndiaFactory();
f.createPayment().pay(1000 + f.createTax().tax(1000));
```

Part 11: Memory, Garbage Collection and OOP Internals

Q121. Where are objects stored in Java memory?

Objects are stored in the Heap memory, while references to those objects (local variables) are stored in the Stack.

```
void test() {
    int count = 5;                      // primitive local: STACK
    Ride r = new Ride();                // reference 'r': STACK  |  Ride object: HEAP
}
```

Q122. What is Garbage Collection in Java?

Garbage Collection is the automatic process by which the JVM reclaims memory occupied by objects that are no longer reachable/referenced by the program.

Q123. Can we force Garbage Collection in Java?

Not directly. We can request it using `System.gc()`, but the JVM decides whether and when to actually perform garbage collection.

Q124. What is the difference between Stack and Heap memory in Java?

The Stack stores method calls, local variables, and references, and follows a LIFO structure with fast access. The Heap stores actual objects and is managed by the garbage collector, with a larger but relatively slower memory area.

Aspect	Stack	Heap
Stores	Method frames, local variables, references	Objects, arrays
Scope	Per thread	Shared by all threads
Lifetime	Until method returns	Until garbage collected
Error when full	StackOverflowError	OutOfMemoryError
Speed	Faster	Slower

Q125. What happens when an object has no references pointing to it?

It becomes eligible for garbage collection, and the JVM will reclaim its memory at some point during a GC cycle.

```
Ride r = new Ride();     // object created
r = null;                // no reference left -> eligible for GC
r = new Ride();          // old object now unreachable; new one reachable
```

Q126. What is Memory Leak in the context of Java OOP?

Even though Java has automatic garbage collection, a memory leak can occur when objects are unintentionally kept reachable (e.g., through static references or unclosed resources), preventing the garbage collector from reclaiming them.

```
class SessionCache {
    private static final Map<String, UserSession> CACHE = new HashMap<>();   // lives forever
    static void login(String id, UserSession s) { CACHE.put(id, s); }        // never removed -> LEAK
    // Fix: remove on logout, or use a bounded/expiring cache (Caffeine, LinkedHashMap with
        // removeEldestEntry)
}
```

Part 12: Miscellaneous and Tricky OOP Questions

Q127. Can we instantiate an Abstract class?

No, an abstract class cannot be instantiated directly; it must be subclassed, and the subclass must be instantiated instead.

Q128. Can an Interface have a constructor?

No, interfaces cannot have constructors because they cannot maintain state (instance fields) and are not meant to be instantiated directly.

Q129. What happens if a subclass does not implement all abstract methods of its parent abstract class?

The subclass must also be declared abstract; otherwise, a compile-time error occurs.

Q130. Can a class extend multiple classes in Java?

No, Java does not support multiple inheritance with classes. A class can only extend one superclass but can implement multiple interfaces.

Q131. What is the diamond problem, and how does Java handle it with interfaces?

The diamond problem arises when a class inherits from two sources that both define a method with the same signature, causing ambiguity. Java handles this in interfaces by requiring the implementing class to explicitly override the conflicting default method. (Code: see Q54.)

Q132. Is Java a fully Object-Oriented language?

No, Java is not considered "purely" object-oriented because it uses primitive data types (int, char, boolean, etc.) that are not objects, unlike languages such as Smalltalk.

Q133. What is the difference between Instance Variables and Local Variables?

Instance variables are declared within a class but outside any method, and each object has its own copy; they get default values if not initialized. Local variables are declared within a method and exist only during that method's execution; they must be explicitly initialized before use.

```
class Demo {
    int count;                          // instance variable: default 0
    void test() {
        int x;                          // local variable
        // System.out.println(x);       // COMPILE ERROR: might not have been initialized
        System.out.println(count);      // 0
    }
}
```

Q134. What is the difference between Instance Methods and Static Methods?

Instance methods operate on a specific object and can access instance variables directly. Static methods belong to the class, can be called without creating an object, and cannot directly access instance variables.

Q135. What is a Package in Java?

A package is a namespace that groups related classes and interfaces together, helping to avoid naming conflicts and control access.

> Typical layered packages in a Spring Boot project: `com.shop.controller`, `com.shop.service`, `com.shop.repository`, `com.shop.model`.

Q136. What is the difference between import and package statements?

`package` declares which package the current class belongs to (must be the first line in the file). `import` is used to bring in classes from other packages so they can be used without fully qualified names.

Q137. What is an Enum in Java, and how does OOP apply to it?

An enum is a special data type that represents a fixed set of constants. In Java, enums are actually implemented as classes, meaning each enum constant is an object, and enums can have fields, constructors, and methods.

> **Real-world: Netflix plans**
> Each plan is a fixed constant, but carries its own data and behavior.

```
enum SubscriptionPlan {
    MOBILE(149, 1, "480p"), BASIC(199, 1, "720p"), STANDARD(499, 2, "1080p"), PREMIUM(649, 4, "4K");

    private final int pricePerMonth, screens; private final String quality;
    SubscriptionPlan(int price, int screens, String quality) {
        this.pricePerMonth = price; this.screens = screens; this.quality = quality;
    }
    int yearlyCost() { return pricePerMonth * 12; }
    String quality() { return quality; }
}
System.out.println(SubscriptionPlan.PREMIUM.quality());     // 4K
System.out.println(SubscriptionPlan.BASIC.yearlyCost());    // 2388
```

Q138. Can an Enum implement an Interface in Java?

Yes, an enum can implement one or more interfaces, though it cannot extend another class since it implicitly extends `java.lang.Enum`.

Q139. What is the difference between == and equals() when comparing Enums?

Since each enum constant is a singleton instance, both `==` and `equals()` produce the same result when comparing enum values, but using `==` is generally preferred for enums (null-safe and compile-time type checked).

Q140. What is a Functional Interface in Java?

A functional interface is an interface with exactly one abstract method (though it may have multiple default/static methods), used primarily to support lambda expressions (e.g., `Runnable`, `Comparator`).

```
@FunctionalInterface
interface DiscountRule { double apply(double price); }       // exactly one abstract method

DiscountRule tenPercent = price -> price * 0.9;               // lambda
System.out.println(tenPercent.apply(1000));                   // 900.0
// Built-ins: Predicate<T>, Function<T,R>, Consumer<T>, Supplier<T>, Runnable, Comparator<T>
```

Q141. What is a Lambda Expression in Java, and how does it relate to OOP?

A lambda expression is a concise way to implement a functional interface's single abstract method inline, without writing an anonymous class. It supports functional-style programming while still operating within Java's object-oriented type system.

```
// 1. Anonymous class (verbose)
Runnable r1 = new Runnable() {
    @Override public void run() { System.out.println("Sending email"); }
};
// 2. Lambda (same thing, one line)
Runnable r2 = () -> System.out.println("Sending email");
// 3. Method reference
Runnable r3 = EmailService::sendPending;      // assumes: static void sendPending()
new Thread(r2).start();
```

Q142. What is Autoboxing and Unboxing in Java?

Autoboxing is the automatic conversion of a primitive type into its corresponding wrapper object (e.g., `int` to `Integer`). Unboxing is the reverse process, converting a wrapper object back into its primitive type.

```
Integer boxed = 10;          // autoboxing   (int -> Integer)
int plain = boxed;           // unboxing     (Integer -> int)
```

```
Integer a = 127, b = 127;    System.out.println(a == b);   // true  (Integer cache -128..127)
Integer c = 128, d = 128;    System.out.println(c == d);   // false (different objects!) -> use equals()
Integer n = null;  int bad = n;                             // NullPointerException on unboxing
```

Q143. What is the difference between an Object reference and the Object itself?

The object itself is the actual data stored in the heap. The reference is a variable that holds the memory address pointing to that object. Multiple references can point to the same object.

Q144. What is the difference between Pass by Value and Pass by Reference in Java (in the context of objects)?

Java is strictly pass-by-value. When an object is passed to a method, the value of the reference (the memory address) is copied, so the method can modify the object's internal state, but reassigning the reference inside the method does not affect the original reference outside.

```
static void modify(Ride r) {
    r.fare = 999;              // changes the SAME object -> visible outside
    r = new Ride();            // reassigns only the local copy of the reference
    r.fare = 1;                // does NOT affect caller
}
Ride ride = new Ride(); ride.fare = 100;
modify(ride);
System.out.println(ride.fare);  // 999
```

Q145. What is the difference between an Interface's default method and an Abstract class's concrete method?

Both provide a method with an implementation. However, a default method exists in an interface and is meant to provide optional/shared behavior across unrelated implementing classes, while a concrete method in an abstract class is part of a more specific inheritance hierarchy and can access instance state directly. (Comparison table: see Q52.)

Q146. What is Object Cloning, and what are the risks of a shallow clone?

Object cloning creates a copy of an object using the `clone()` method. A shallow clone risks unintended side effects because reference-type fields in the cloned object still point to the same underlying objects as the original, so changes to those shared objects affect both copies. (Code: see Q72.)

Q147. What is the difference between a Concrete Class and an Abstract Class?

A concrete class provides implementations for all its methods and can be instantiated directly. An abstract class may have unimplemented (abstract) methods and cannot be instantiated directly.

Q148. Why are Strings immutable in Java, and how does this relate to OOP design principles?

Strings are immutable for security, thread safety, and to allow safe caching (via the String pool). This is an example of designing a class to protect its internal state, similar to the goals of encapsulation.

Q149. What is the difference between == and .equals() when comparing Strings?

`==` compares whether two references point to the same object in memory. `.equals()` (overridden in the String class) compares the actual character sequence/content of the strings.

```
String a = "hello";
String b = "hello";                 // same literal -> same object in String pool
String c = new String("hello");     // new object on heap

System.out.println(a == b);         // true   (same pooled object)
System.out.println(a == c);         // false  (different objects)
System.out.println(a.equals(c));    // true   (same content)
System.out.println(a == c.intern());// true   (intern() returns the pooled one)
```

Q150. What is the difference between an IS-A relationship and a HAS-A relationship in OOP?

IS-A represents inheritance (a Dog IS-A Animal). HAS-A represents composition/aggregation (a Car HAS-A Engine). Choosing the right relationship is fundamental to good object-oriented design.

Q151. What is the role of Polymorphism in achieving loose coupling?

Polymorphism allows code to work with a general reference type (e.g., an interface or superclass) rather than a specific concrete class, so implementations can change or be extended without modifying the code that uses them.

Q152. What is Object Slicing, and does it apply to Java?

Object slicing (losing subclass-specific data when copying an object as its base type) is a concept mainly relevant to languages like C++ with value semantics. In Java, since objects are always accessed via references, slicing does not occur in the same way.

Part 13: Modern Java Features (Java 8 to 17)

Q153. What are the features of Java 8?

- Static and default methods in Interface.
- Lambda Expressions.
- New Date and Time API (replaced the old flawed `Date`/`Calendar`).
- Also commonly asked: Stream API, Optional, Method References, Functional Interfaces (`Predicate`, `Function`, `Consumer`, `Supplier`).

> **Real-world: Netflix**
> Find top-rated English movies, sorted by rating, as titles only - with streams and lambdas.

```
record Title(String name, String lang, double rating) {}      // record: Java 16 (used here for brevity)

List<Title> catalog = List.of(
    new Title("Leo", "Tamil", 7.4), new Title("Dark", "German", 8.7),
    new Title("Narcos", "English", 8.8), new Title("Ozark", "English", 8.5));

List<String> top = catalog.stream()
    .filter(t -> t.lang().equals("English"))                // intermediate
    .filter(t -> t.rating() > 8.0)
    .sorted(Comparator.comparingDouble(Title::rating).reversed())
    .map(Title::name)
    .collect(Collectors.toList());                          // terminal
System.out.println(top);                                    // [Narcos, Ozark]

Optional<Title> best = catalog.stream().max(Comparator.comparingDouble(Title::rating));
best.ifPresent(t -> System.out.println(t.name()));          // Narcos
LocalDate release = LocalDate.of(2026, 10, 12);             // new Date-Time API
```

Q154. What are the features of Java 11?

- New String methods - `isBlank()`, `lines()`, `repeat()`, `strip()`, `stripLeading()`, `stripTrailing()`.
- New File methods - `Files.readString()` and `Files.writeString()` static methods from the `Files` class.
- Also commonly asked: `var` in lambda parameters, the standard `HttpClient`, `Optional.isEmpty()`, running a single source file with `java Hello.java`.

```
"   ".isBlank();                      // true
"ab".repeat(3);                       // ababab
"  hi  ".strip();                     // "hi"   (Unicode-aware, better than trim())

Path p = Path.of("orders.txt");
Files.writeString(p, "Order#101 PAID");
String text = Files.readString(p);    // "Order#101 PAID"
```

> Newer LTS features worth knowing: Java 14-16 `record` (immutable data class), Java 15 text blocks (""" ... """), Java 17 `sealed` classes, Java 21 virtual threads and pattern matching for switch.

Part 14: Quick Revision Cheat Sheets

OOP concept to keyword map

Concept	Java keyword / feature	Remember
Inheritance	extends, super	Single class only; interfaces can be many
Polymorphism	@Override, overloading	Overriding = runtime; overloading = compile time
Encapsulation	private + getters/setters	Validate inside setters
Abstraction	abstract, interface	Cannot instantiate abstract class or interface
Immutability	final class, final fields	String, Integer, LocalDate are immutable
Composition	field of another class	Prefer over inheritance

What cannot be overridden

Member	Why
static method	Belongs to class (it is hidden, not overridden)
private method	Not visible to subclass
final method	Explicitly forbidden
Constructor	Not inherited
Field/variable	Fields are never polymorphic; reference type decides

Exception hierarchy at a glance

Type	Examples	Must handle at compile time?
Error	OutOfMemoryError, StackOverflowError	No (do not catch)
Checked (Exception)	IOException, SQLException, custom extends Exception	Yes
Unchecked (RuntimeException)	NullPointerException, IllegalArgumentException, ArithmeticException	No

Collections quick pick

Need	Use
Ordered, duplicates allowed, fast read	ArrayList
Fast insert/remove at both ends (queue/deque)	ArrayDeque / LinkedList
No duplicates, no order	HashSet
No duplicates, insertion order	LinkedHashSet
No duplicates, sorted	TreeSet
Key-value, fast lookup	HashMap
Key-value, sorted by key	TreeMap
Key-value, thread-safe	ConcurrentHashMap

Part 15: Real-World Case Studies

Three end-to-end mini designs. Each one maps every OOP pillar and design pattern you studied to a system you use daily. In interviews, picking one of these as your "go-to example" makes your answers concrete and memorable.

Case Study 1: Uber (Ride Booking)

OOP concept	Where it appears in Uber
Encapsulation	`Wallet.balance`, `Driver.rating` are private; changed only via validated methods
Inheritance	`Ride` -> `AutoRide`, `PremiumRide`, `BikeRide`
Polymorphism	`ride.calculateFare(km)` behaves differently per ride type
Abstraction	`PaymentMethod` interface hides UPI/Card/Wallet details
Composition	`Ride` HAS-A `Driver`, `Rider`, `PricingStrategy`
Strategy pattern	Normal / Night / Surge pricing
Factory pattern	`RideFactory.create("AUTO")`
Observer pattern	Rider app, SMS, and tracker are notified on status change
Singleton	`RideMatchingService`, `AppConfig`
Custom exception	`InsufficientBalanceException`, `NoDriverAvailableException`

```
// ---------- Abstraction + Strategy ----------
interface FareStrategy { double fare(double km); }
class NormalFare implements FareStrategy { public double fare(double km) { return 30 + km * 12; } }
class SurgeFare  implements FareStrategy {
    private final double m;  SurgeFare(double m) { this.m = m; }
    public double fare(double km) { return (30 + km * 12) * m; }
}

// ---------- Observer ----------
enum RideStatus { REQUESTED, ACCEPTED, ARRIVED, STARTED, COMPLETED }
interface RideListener { void onStatusChange(String rideId, RideStatus status); }
class RiderApp   implements RideListener { public void onStatusChange(String id, RideStatus s) {
    System.out.println("App: " + id + " " + s); } }
class SmsService implements RideListener { public void onStatusChange(String id, RideStatus s) {
    System.out.println("SMS: " + id + " " + s); } }

// ---------- Inheritance + Polymorphism + Composition ----------
abstract class Vehicle {
    protected final String plate;
    Vehicle(String plate) { this.plate = plate; }
    abstract int capacity();
}
class Auto extends Vehicle { Auto(String p) { super(p); } int capacity() { return 3; } }
class Sedan extends Vehicle { Sedan(String p) { super(p); } int capacity() { return 4; } }

class Driver {
    private final String name; private final Vehicle vehicle; private double rating = 5.0;
    Driver(String name, Vehicle vehicle) { this.name = name; this.vehicle = vehicle; }
    String getName() { return name; }
}

class Ride {
    private final String id;
    private final Driver driver;                          // composition / association
    private FareStrategy fareStrategy;                    // composition (swappable)
    private RideStatus status = RideStatus.REQUESTED;
    private final List<RideListener> listeners = new ArrayList<>();

    Ride(String id, Driver driver, FareStrategy fs) { this.id = id; this.driver = driver;
        this.fareStrategy = fs; }
    void addListener(RideListener l) { listeners.add(l); }
    void updateStatus(RideStatus s) {
```

```
        this.status = s;
        listeners.forEach(l -> l.onStatusChange(id, s));    // notify observers
    }
    double fare(double km) { return fareStrategy.fare(km); }
    void setFareStrategy(FareStrategy fs) { this.fareStrategy = fs; }
}

// ---------- Run it ----------
Driver d = new Driver("Kumar", new Sedan("TN-37-XY-4321"));
Ride ride = new Ride("R1001", d, new NormalFare());
ride.addListener(new RiderApp());  ride.addListener(new SmsService());

ride.updateStatus(RideStatus.ACCEPTED);                    // App: R1001 ACCEPTED / SMS: R1001 ACCEPTED
ride.setFareStrategy(new SurgeFare(1.5));                  // it started raining
System.out.println(ride.fare(10));                         // (30 + 120) * 1.5 = 225.0
```

Case Study 2: E-commerce (Amazon / Flipkart style)

OOP concept	Where it appears in E-commerce
Encapsulation	`Product.price`, `stock` are private; `reduceStock(qty)` validates
Inheritance	`Product` -> `Electronics`, `Clothing`, `Grocery`
Polymorphism	`product.shippingCost()` differs (Grocery: cold chain, Electronics: insured)
Abstraction	`PaymentMethod`, `Discount` interfaces
Composition	`Order` HAS-A `OrderItem`s (they die with the order)
Aggregation	`Cart` references `Product`s that exist independently
equals/hashCode	Same SKU = same product in Set/Map
Builder	Complex `Shipment` / `Order` creation
Decorator	Gift wrap, warranty, express delivery
Adapter	Third-party payment SDK to our `PaymentGateway`
Dependency Inversion	`OrderService(PaymentGateway, Discount)` via Spring DI

```
// ---------- Encapsulation + Inheritance + Polymorphism ----------
abstract class Product {
    private final String sku; private final String name;
    private double price; private int stock;
    Product(String sku, String name, double price, int stock) {
        this.sku = sku; this.name = name; setPrice(price); this.stock = stock;
    }
    public void setPrice(double p) { if (p <= 0) throw new IllegalArgumentException("Bad price");
        this.price = p; }
    public double getPrice() { return price; }
    public String getSku()   { return sku; }
    public void reduceStock(int q) {
        if (q > stock) throw new IllegalStateException("Out of stock: " + name);
        stock -= q;
    }
    abstract double shippingCost();                                  // polymorphic
    @Override public boolean equals(Object o) { return o instanceof Product p && sku.equals(p.sku); }
    @Override public int hashCode() { return sku.hashCode(); }
}
class Electronics extends Product {
    Electronics(String s, String n, double p, int st) { super(s, n, p, st); }
    double shippingCost() { return 0; }                              // free + insured
}
class Grocery extends Product {
    Grocery(String s, String n, double p, int st) { super(s, n, p, st); }
    double shippingCost() { return 40; }                             // cold-chain fee
}

// ---------- Composition ----------
class OrderItem {
    final Product product; final int qty;
    OrderItem(Product p, int q) { product = p; qty = q; p.reduceStock(q); }
    double total() { return product.getPrice() * qty + product.shippingCost(); }
}
```

```
class Order {
    private final List<OrderItem> items = new ArrayList<>();
    void add(Product p, int q) { items.add(new OrderItem(p, q)); }
    double subtotal() { return items.stream().mapToDouble(OrderItem::total).sum(); }
}

// ---------- Dependency Inversion + Strategy-like Discount ----------
interface PaymentGateway { boolean charge(double amount); }
interface Discount { double apply(double amount); }

class OrderService {
    private final PaymentGateway gateway; private final Discount discount;
    OrderService(PaymentGateway g, Discount d) { this.gateway = g; this.discount = d; }
    boolean checkout(Order order) {
        double payable = discount.apply(order.subtotal());
        return gateway.charge(payable);
    }
}

// ---------- Run it ----------
Product phone = new Electronics("SKU-PH1", "Phone", 20000, 10);
Product rice  = new Grocery("SKU-RC1", "Rice 10kg", 650, 100);
Order order = new Order();
order.add(phone, 1);  order.add(rice, 2);                  // 20000 + (1300 + 40) = 21340
OrderService svc = new OrderService(amt -> { System.out.println("Charged " + amt); return true; },
                                    amt -> amt * 0.90);     // 10% festival discount
svc.checkout(order);                                        // Charged 19206.0
```

Case Study 3: Netflix (Streaming Platform)

OOP concept	Where it appears in Netflix
Abstraction	abstract `Content` with `totalDurationMinutes()`
Inheritance	`Content` -> `Movie`, `Series`, `Documentary`
Polymorphism	`List<Content>` plays any type; duration computed per type
Interface	`Playable`, `Downloadable` (capabilities); default `skipIntro()`
Enum	`SubscriptionPlan` with price, screens, quality
Comparable / Comparator	Sort by rating (natural), by year or title (custom)
Aggregation	`Playlist` / `MyList` holds `Content` that exists independently
Static	`Subscriber.totalSubscribers` counter
Singleton	`CatalogService`
Observer	New episode notifications to subscribers
Strategy	Recommendation algorithms (trending, because-you-watched)
Java 8 streams	Filtering and grouping the catalog

```
// ---------- Abstraction + Inheritance ----------
abstract class Content implements Comparable<Content> {
    protected final String title; protected final String genre;
    protected final double rating; protected final int year;
    Content(String t, String g, double r, int y) { title = t; genre = g; rating = r; year = y; }
    abstract int totalDurationMinutes();
    @Override public int compareTo(Content o) { return Double.compare(o.rating, rating); }   // best first
    @Override public String toString() { return title + " (" + rating + ")"; }
}
class Movie extends Content {
    private final int minutes;
    Movie(String t, String g, double r, int y, int m) { super(t, g, r, y); minutes = m; }
    int totalDurationMinutes() { return minutes; }
}
class Series extends Content {
    private final int episodes, perEpisode;
    Series(String t, String g, double r, int y, int e, int p) { super(t, g, r, y); episodes = e;
        perEpisode = p; }
    int totalDurationMinutes() { return episodes * perEpisode; }
}
```

```
// ---------- Singleton (enum) ----------
enum CatalogService {
    INSTANCE;
    private final List<Content> catalog = new ArrayList<>();
    void add(Content c) { catalog.add(c); }
    List<Content> all() { return Collections.unmodifiableList(catalog); }      // encapsulation
}

// ---------- Strategy: recommendations ----------
interface RecommendationStrategy { List<Content> recommend(List<Content> catalog); }
class TopRated implements RecommendationStrategy {
    public List<Content> recommend(List<Content> cat) {
        return cat.stream().sorted().limit(3).collect(Collectors.toList());
    }
}
class ByGenre implements RecommendationStrategy {
    private final String genre;  ByGenre(String g) { genre = g; }
    public List<Content> recommend(List<Content> cat) {
        return cat.stream().filter(c -> c.genre.equals(genre)).collect(Collectors.toList());
    }
}

// ---------- Run it ----------
CatalogService cs = CatalogService.INSTANCE;
cs.add(new Movie("Leo", "Action", 7.4, 2023, 164));
cs.add(new Series("Dark", "Thriller", 8.7, 2017, 26, 55));
cs.add(new Series("Narcos", "Crime", 8.8, 2015, 30, 50));

RecommendationStrategy rec = new TopRated();
System.out.println(rec.recommend(cs.all()));              // [Narcos (8.8), Dark (8.7), Leo (7.4)]
rec = new ByGenre("Thriller");
System.out.println(rec.recommend(cs.all()));              // [Dark (8.7)]

Map<String, List<Content>> byGenre =
    cs.all().stream().collect(Collectors.groupingBy(c -> c.genre));     // group by genre
```

Interview tip: when asked "explain OOP / polymorphism / SOLID / design patterns", pick ONE of these three systems and stay with it through your whole answer. Interviewers love a consistent real-world story.

Part 16: Top 50 Java OOP Interview Questions

Most frequently asked in campus placements, service companies and product-company screening rounds. Answers are written the way you would say them in an interview: definition first, then a one-line example.

A. Core OOP (1-15)

Q1. Explain the four pillars of OOP with a real-world example.

Use Uber: **Encapsulation** - wallet balance is private, changed only via `pay()`. **Inheritance** - `Car`, `Auto`, `Bike` extend `Vehicle`. **Polymorphism** - `calculateFare()` behaves differently per ride type. **Abstraction** - rider calls `bookRide()` without knowing the matching algorithm.

Q2. Abstract class vs Interface - when do you use which?

Use an **abstract class** when related classes share state or common code (`Content` -> `Movie`, `Series`). Use an **interface** to define a capability across unrelated classes (`Payable`, `Downloadable`). A class can extend one abstract class but implement many interfaces. Since Java 8, interfaces can have default/static methods, but still no instance state or constructors.

Q3. Method overloading vs overriding?

Overloading: same name, different parameters, same class, resolved at compile time. Overriding: same signature in child class, resolved at runtime via dynamic dispatch. Overriding cannot reduce visibility, and cannot apply to static, private or final methods.

Q4. Why is multiple inheritance not supported with classes? How do interfaces solve it?

Because of the diamond problem: two parents with the same method make the call ambiguous. A class may implement many interfaces; if two have the same default method, the class must override it and can call `A.super.method()`.

Q5. Composition vs Inheritance - which is better?

Prefer composition ("favor composition over inheritance"). Inheritance creates tight coupling and the fragile base class problem; composition lets you change behavior at runtime (e.g., swap a `PricingStrategy`). Use inheritance only for a genuine IS-A relationship.

Q6. Aggregation vs Composition?

Both are HAS-A. In composition the part cannot outlive the whole (Order - OrderItem). In aggregation the part exists independently (Playlist - Movie).

Q7. What is the difference between Encapsulation and Abstraction?

Encapsulation hides data (private fields + controlled access). Abstraction hides complexity (exposes what, hides how, through abstract classes and interfaces).

Q8. What is the Liskov Substitution Principle? Give a violation.

A subclass must be usable wherever the parent is expected without breaking correctness. Violation: `Penguin extends Bird` and throws an exception in `fly()`. Fix: split into `Flyable` capability.

Q9. Explain SOLID with one line each.

S: one reason to change. O: extend by adding code, not editing (new `Discount` class). L: subclasses are substitutable. I: many small interfaces. D: depend on abstractions (`OrderService` -> `PaymentGateway`).

Q10. What is the difference between Dependency Injection and Dependency Inversion?

Dependency **Inversion** is the principle: depend on abstractions. Dependency **Injection** is a technique to supply those dependencies from outside (constructor injection). Spring's IoC container performs the injection.

Q11. What is the difference between tight and loose coupling? How does Spring achieve loose coupling?

Tight: class creates and depends on concrete classes via `new`. Loose: class depends on interfaces. Spring achieves it through IoC + DI: you declare `@Service` / `@Autowired` (or constructor injection) and Spring wires implementations.

Q12. What is polymorphism? Compile-time vs runtime?

One interface, many behaviors. Compile-time = overloading. Runtime = overriding, where the JVM picks the method based on the actual object: `Ride r = new AutoRide(); r.calculateFare()` runs AutoRide's version.

Q13. What is upcasting and downcasting? When does ClassCastException occur?

Upcast (child to parent) is implicit and safe. Downcast (parent to child) is explicit and fails with `ClassCastException` when the real object is not that child. Guard it with `instanceof`.

Q14. Can we override static or private methods? Can we overload them?

Static and private methods cannot be overridden (static is hidden, private is invisible). Both can be overloaded.

Q15. What is the use of the super and this keywords?

`this` = current object (disambiguate fields, call another constructor, pass self). `super` = parent (call parent constructor, access parent members, invoke overridden method). Both `this()` and `super()` must be the first statement of a constructor.

B. Object class, Strings and Immutability (16-25)

Q16. Difference between == and equals()?

`==` compares references (or primitive values). `equals()` compares logical content when overridden. Example: `new String("a") == new String("a")` is false, but `equals` is true.

Q17. What is the equals() and hashCode() contract?

If `a.equals(b)` then `a.hashCode() == b.hashCode()`. Equal hash codes do not imply equal objects. Breaking it causes duplicates in `HashSet` and lookup failures in `HashMap`.

Q18. Why is String immutable? What are the benefits?

Security (class names, URLs, DB credentials cannot be altered), thread safety, String pool caching, and a safe, cached `hashCode()` which makes it a perfect HashMap key.

Q19. String vs StringBuilder vs StringBuffer?

String: immutable. StringBuilder: mutable, not thread-safe, fastest. StringBuffer: mutable, thread-safe (synchronized), slower. Use StringBuilder for loops that concatenate.

Q20. What is the String pool? How many objects does `new String("abc")` create?

The pool is a heap area that stores unique string literals. `new String("abc")` creates one heap object, plus one pooled literal "abc" if it was not already there (so 1 or 2).

Q21. How do you create an immutable class?

Make the class `final`, all fields `private final`, no setters, initialize via constructor, and defensively copy mutable fields in and out (e.g., `List.copyOf()`, `new Date(d.getTime())`). Or use a `record` (Java 16).

```
public final class Money {
    private final long amountInPaise;
    private final String currency;
    public Money(long amountInPaise, String currency) { this.amountInPaise = amountInPaise; this.currency
        = currency; }
    public Money add(Money o) { return new Money(amountInPaise + o.amountInPaise, currency); }   //
        // returns new object
}
```

Q22. Shallow copy vs deep copy?

Shallow copies references (both objects share nested objects). Deep copies nested objects too. Implement deep copy with a copy constructor or serialization, rather than relying on `clone()`.

Q23. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the reference value is copied: the method can mutate the object but cannot make the caller's variable point elsewhere.

Q24. What is the Integer cache? What does `Integer a=128, b=128; a==b` print?

Integers from -128 to 127 are cached, so `==` is true there. For 128 it prints `false` because they are different objects. Always compare wrappers with `equals()`.

Q25. Why is the main method static?

So the JVM can call it without creating an object of the class. Signature: `public static void main(String[] args)` - public (JVM access), static (no object), void (nothing returned), String[] (command-line args).

C. Exceptions, Generics and Collections (26-37)

Q26. Checked vs unchecked exceptions? Which to use for custom exceptions?

Checked (extends Exception) are enforced by the compiler for recoverable situations (IOException). Unchecked (extends RuntimeException) represent programming errors or unrecoverable conditions. Modern frameworks like Spring prefer unchecked custom exceptions to keep signatures clean.

Q27. final vs finally vs finalize?

`final` - modifier (constant, no override, no inherit). `finally` - block that always runs after try/catch. `finalize()` - GC hook, deprecated.

Q28. What happens if both try and finally have a return statement?

The `finally` return wins and overrides the try's return value (and swallows exceptions). Never return from finally.

```
static int test() {
    try { return 1; } finally { return 2; }     // returns 2
}
```

Q29. Can finally be skipped?

Only if the JVM exits (`System.exit()`), crashes, or the thread is killed before reaching it.

Q30. Explain try-with-resources.

Any `AutoCloseable` declared in the try parentheses is closed automatically in reverse order, even when exceptions occur. Suppressed exceptions are attached to the primary one.

Q31. How does HashMap work internally?

Key's `hashCode()` is spread and mapped to a bucket index (array of nodes). Collisions are stored as a linked list; when a bucket reaches 8 entries (and table size >= 64) it becomes a red-black tree (Java 8+). `get()` finds the bucket, then uses `equals()` to find the key. Default capacity 16, load factor 0.75, resizes (doubles) when exceeded.

Q32. HashMap vs Hashtable vs ConcurrentHashMap?

HashMap: not thread-safe, allows one null key. Hashtable: legacy, synchronized on every method, no nulls. ConcurrentHashMap: thread-safe with fine-grained locking/CAS, no null keys or values, best choice for concurrency.

Q33. ArrayList vs LinkedList?

ArrayList: array-backed, O(1) get, O(n) middle insert. LinkedList: doubly linked, O(1) insert/remove at known node, O(n) get. In practice ArrayList wins for most workloads.

Q34. HashSet vs TreeSet vs LinkedHashSet?

HashSet: no order, O(1). LinkedHashSet: insertion order, O(1). TreeSet: sorted, O(log n), needs Comparable/Comparator.

Q35. Comparable vs Comparator?

Comparable: natural order inside the class (`compareTo`), one ordering. Comparator: external, many orderings (`Comparator.comparing(Film::getYear)`).

Q36. Fail-fast vs fail-safe iterators?

Fail-fast (ArrayList, HashMap) throw `ConcurrentModificationException` if modified during iteration. Fail-safe (CopyOnWriteArrayList, ConcurrentHashMap) iterate over a snapshot or tolerate changes.

Q37. What is type erasure? What are wildcards?

Generic types are erased at compile time to their bounds, so `List<String>` is `List` at runtime. Wildcards: `? extends T` (read-only producer), `? super T` (consumer), `?` (unknown). Rule: PECS - Producer Extends, Consumer Super.

D. Design Patterns and Modern Java (38-45)

Q38. Implement a thread-safe Singleton. Which is the best way?

Best: Enum singleton (reflection and serialization safe). Next best: Bill Pugh holder, or double-checked locking with `volatile`. See Q94 for all six implementations. Know why `volatile` is needed: it prevents instruction reordering that could expose a half-constructed object.

Q39. How can Singleton be broken?

Reflection (call the private constructor via `setAccessible(true)`), serialization/deserialization (creates a new instance unless `readResolve()` is defined), cloning, and multiple class loaders. Enum singleton is immune to the first three.

Q40. Factory vs Abstract Factory vs Builder?

Factory: one method returns one product type by input (`RideFactory`). Abstract Factory: creates a family of related products (`IndiaFactory`, `UsFactory`). Builder: constructs one complex object step by step (`Shipment.Builder`).

Q41. Strategy vs State vs Observer - when to use?

Strategy: swap an algorithm chosen by the client (surge pricing). State: object changes behavior as its own state changes (Ride: REQUESTED -> STARTED -> COMPLETED). Observer: one-to-many notification (new episode alerts).

Q42. What is a functional interface? Name built-in ones.

Interface with exactly one abstract method, usable with lambdas. `Predicate<T>` (boolean test), `Function<T,R>` (transform), `Consumer<T>` (accept, no return), `Supplier<T>` (no input, returns), `Runnable`, `Comparator<T>`.

Q43. Intermediate vs terminal operations in Streams?

Intermediate ops (`filter`, `map`, `sorted`) are lazy and return a stream. Terminal ops (`collect`, `forEach`, `count`, `reduce`) trigger processing and end the pipeline. A stream cannot be reused after a terminal operation.

Q44. What is Optional and why use it?

A container that may or may not hold a value, making "no value" explicit and avoiding NullPointerException. Use `orElse`, `orElseThrow`, `map`, `ifPresent`. Do not use it for fields or method parameters.

Q45. What are records and sealed classes?

`record Point(int x, int y)` (Java 16) is an immutable data carrier with auto-generated constructor, getters, `equals`, `hashCode`, `toString`. `sealed` classes (Java 17) restrict which classes may extend them using `permits`, enabling exhaustive pattern matching.

E. Memory and JVM (46-50)

Q46. JDK vs JRE vs JVM?

JVM executes bytecode. JRE = JVM + core libraries (runs apps). JDK = JRE + compiler (`javac`) and dev tools.

Q47. Why is Java platform independent?

Source compiles to bytecode (`.class`), and each OS has its own JVM that interprets/JIT-compiles that bytecode. Write once, run anywhere.

Q48. Stack vs Heap? What causes StackOverflowError and OutOfMemoryError?

Stack: per-thread, method frames and local variables. Heap: shared, all objects. Deep or infinite recursion causes `StackOverflowError`; too many live objects or leaks (e.g., an ever-growing static map) cause `OutOfMemoryError`.

Q49. How does garbage collection decide an object is garbage?

By reachability from GC roots (stack variables, static fields, active threads). Unreachable objects are collected, even if they reference each other (cyclic references are fine). Common collectors: G1 (default), ZGC, Parallel.

Q50. What are the steps when you write `new Ride()`?

Class is loaded (if first use, static blocks run) -> memory allocated on heap with default values -> parent constructor chain runs (instance initializers then constructor body, parent first) -> reference returned and assigned to the variable on the stack.

Bonus: Output prediction questions

B1. What is the output?

```
class A { static void show() { System.out.println("A"); }  void print() { System.out.println("A.print");
    } }
class B extends A { static void show() { System.out.println("B"); }  void print() {
    System.out.println("B.print"); } }
A obj = new B();
obj.show();      // prints: A        (static -> reference type)
obj.print();     // prints: B.print  (overridden -> object type)
```

B2. What is the output?

```
String s1 = "java";
String s2 = new String("java");
String s3 = "ja" + "va";                // compile-time constant -> pooled
System.out.println(s1 == s2);           // false
System.out.println(s1 == s3);           // true
System.out.println(s1.equals(s2));      // true
```

B3. What is the output?

```
class Parent {
    Parent() { System.out.println("Parent"); show(); }
    void show() { System.out.println("Parent.show"); }
}
class Child extends Parent {
    int x = 10;
    Child() { System.out.println("Child"); }
    @Override void show() { System.out.println("Child.show x=" + x); }
}
new Child();
// Parent
// Child.show x=0        <- field not yet initialized when parent constructor calls overridden method!
// Child
```

B4. What is the output?

```
try {
    int[] a = new int[2];
    a[5] = 1;
} catch (ArithmeticException e) {
    System.out.println("Arithmetic");
} catch (Exception e) {
    System.out.println("Exception");     // prints this (ArrayIndexOutOfBoundsException is an Exception)
} finally {
    System.out.println("Finally");       // always prints
}
```

Final advice: for every concept, be ready to give (1) a one-line definition, (2) one real-world example (Uber / E-commerce / Netflix), (3) one gotcha or trade-off. That three-part answer pattern is what separates good candidates from average ones. All the best!